

SISTEMA DE CONTROL DE TALLER DE MOTOCICLETAS (MVP)



DONDE LA PASIÓN MECÁNICA
SE UNE A LA INNOVACIÓN DIGITAL.

Contenido

PLANTEAMIENTO DEL PROBLEMA: SISTEMA DE CONTROL DE TALLER DE MOTOCICLETAS (MVP)	3
Contexto del Negocio.....	3
Objetivo General.....	3
Requerimientos del Sistema	3
MODELO DE DOMINIO (atributos y relaciones)	5
CASOS DE USO	6
Caso de uso: CU-001	7
Caso de uso: CU-002.....	8
Caso de uso: CU-003.....	9
Caso de uso: CU-004.....	10
Caso de uso: CU-005.....	11
Mapa Global de Casos de Uso del Sistema	12
DISEÑO DE REGLAS DE NEGOCIO	13
Matriz de Transición de Estado.....	13
Matriz de Control de Acceso por Roles (RBAC)	13
Tabla de Invariantes y Validaciones de Campo	14
Reglas de Negocio Consolidadas (RN).....	14
VALIDACIÓN DEL MODELO	16
DISEÑO DE LA ARQUITECTURA	18
La Arquitectura Global del Sistema.	18
Diseño Técnico de la Arquitectura del Backend.	18
La Estructura Base de Carpetas para Backend	19
Módulo: Iniciar Sesión (CU-005)	19
Módulo: Módulo Órdenes + Ítems (CU-001, CU-002, CU-003, CU-004)	21
Módulo: Motos (CU-002a)	23
Módulo: Cliente (CU-002b)	23
Estrategia Global de Errores	24
Estructura general de carpetas.....	25

DISEÑO DE LA BASE DE DATOS.....	28
Modelo entidad relación	28
Relaciones.....	28
Análisis de Tipos de Datos y Restricciones (Constraints)	29
Script SQL Oficial de Creación (DDL para MySQL).....	30
ARQUITECTURA DE FRONTEND	32
Estructura Físicas de Carpetas en React	33
El Mapa de Pantallas (Relación Pantalla vs Endpoints)	34
La Pieza Clave del Frontend: Manejo Automático del Token JWT	35

PLANTEAMIENTO DEL PROBLEMA: SISTEMA DE CONTROL DE TALLER DE MOTOCICLETAS (MVP)

Contexto del Negocio

Un taller de motocicletas requiere modernizar su proceso operativo, el cual actualmente presenta fallas en la trazabilidad de los mantenimientos, falta de control en los costos de repuestos/mano de obra y una gestión ineficiente del estado de los vehículos en servicio.

Para dar solución a este problema, se requiere el diseño y desarrollo de una plataforma web Full Stack (MVP) que centralice el registro de clientes, vehículos, órdenes de trabajo, control de ítems e historial de reparaciones con autenticación y control de acceso basado en roles (RBAC).

Objetivo General

Construir una solución integral en arquitectura cliente-servidor que permita gestionar todo el ciclo de vida de una orden de trabajo dentro del taller, desde la recepción del vehículo hasta su entrega final, garantizando la seguridad de la información, el cumplimiento de las reglas de negocio y la trazabilidad de cada acción.

Requerimientos del Sistema

Módulo de Seguridad y Autenticación

- **Control de Acceso (RBAC):** La plataforma cuenta con dos niveles de acceso diferenciados:
 - **Administrador (ADMIN):** Control total sobre la gestión de usuarios (creación, activación, asignación de roles), clientes, motocicletas, órdenes de trabajo e ítems, así como la potestad exclusiva para la entrega o cancelación definitiva de órdenes.
 - **Mecánico (MECANICO):** Acceso operativo para consultar órdenes, registrar avances de diagnóstico y trabajo, agregar ítems de mano de obra/repuestos y gestionar la eliminación de sus propios registros.

- **Seguridad:** Autenticación mediante tokens JWT (con ciclo de vida corto y refresco de sesión) y almacenamiento seguro de credenciales utilizando algoritmos de hashing con bcrypt.

Módulo de Gestión de Clientes y Vehículos

- Registro dinámico e individual de clientes y motocicletas asociadas.
- Validación de identificación única por placa de vehículo.
- Mecanismo de registro rápido durante el flujo de apertura de órdenes para evitar interrupciones en la atención al cliente.

Módulo de Órdenes de Trabajo y Cotización

- **Apertura de Orden:** Creación de solicitudes asociadas a un vehículo registrado, especificando el reporte inicial de falla.
- **Gestión de Ítems:** Capacidad de agregar o remover ítems clasificados como **Mano de Obra** o **Repuesto**, definiendo cantidades y costos unitarios.
- **Recálculo Automático:** Actualización en tiempo real del valor total acumulado en la orden en función de los ítems asociados.

Módulo de Flujo de Estados y Auditoría

- **Ciclo de Vida Controlado:** Transición secuencial y lógica del estado del servicio:

RECIBIDA → DIAGNOSTICO → EN_PROCESO → LISTA → ENTREGADA

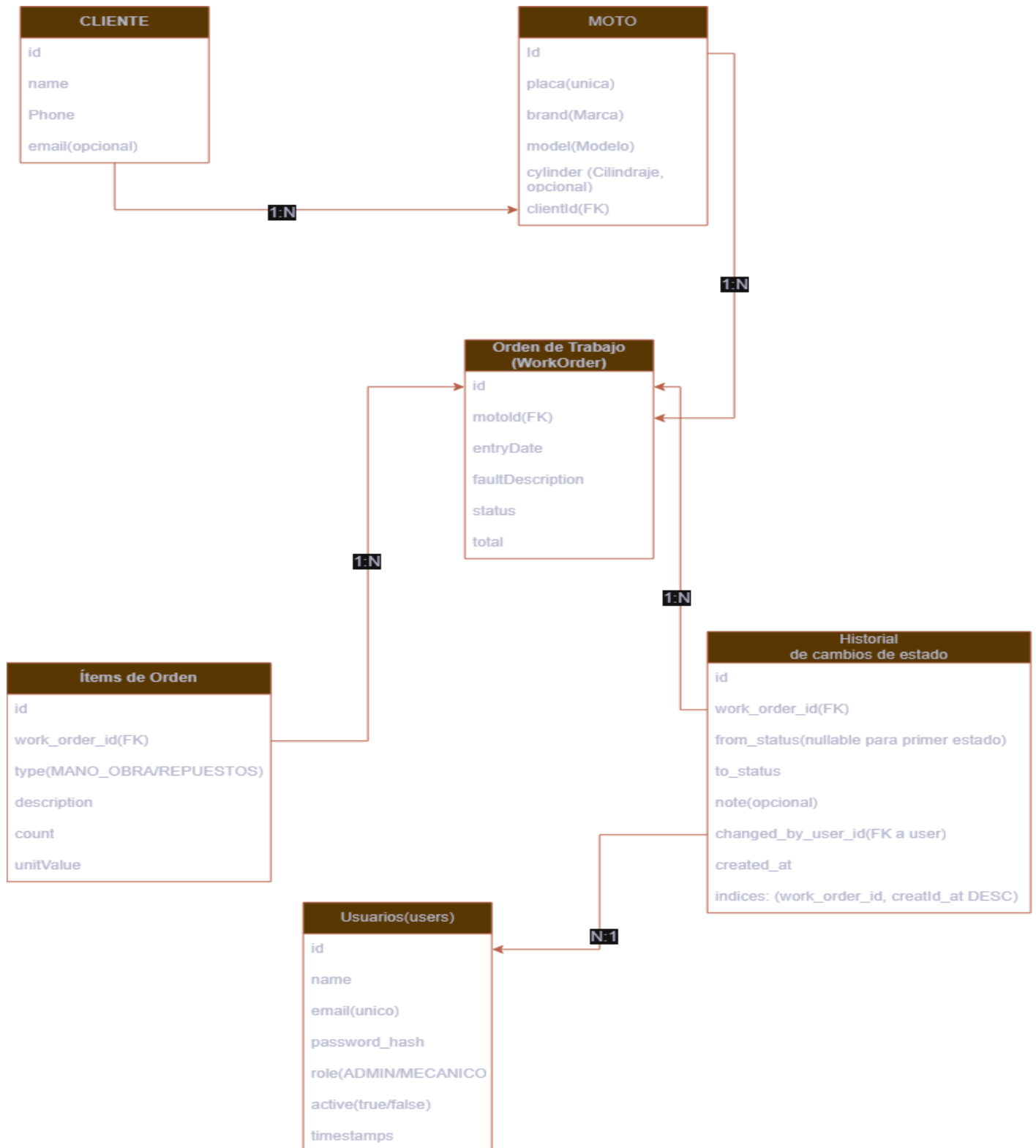
(Permitiendo el estado CANCELADA desde cualquier etapa previa a la entrega).

- **Historial e Inmutabilidad (Auditoría):** Registro automático e inmutable de cada cambio de estado, capturando la fecha y hora exacta, el estado origen y destino, el usuario responsable de la acción y la nota o justificación del cambio.

Stack Tecnológico Sugerido para la Solución

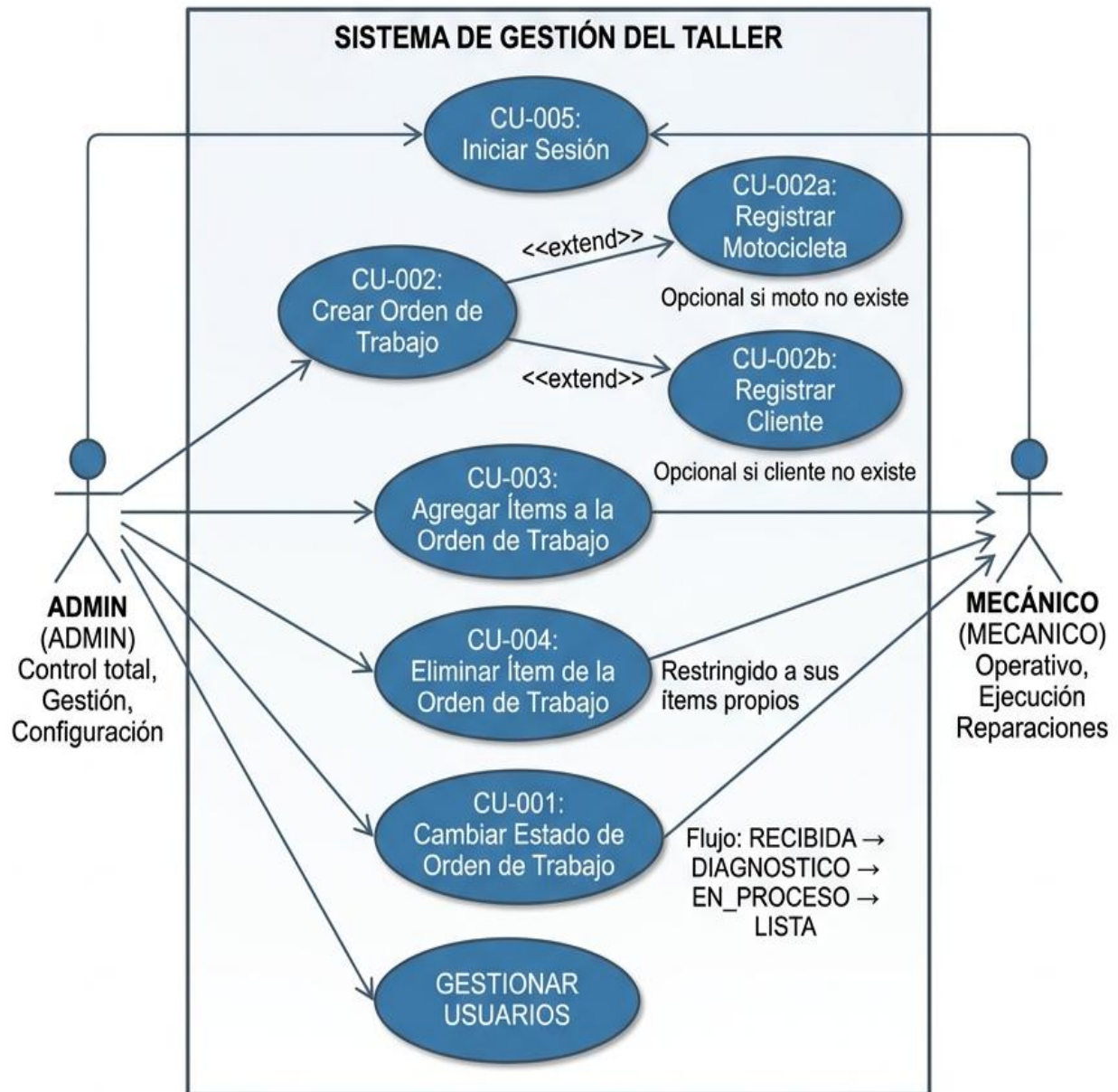
- **Backend:** Node.js, Express.js, Sequelize ORM.
- **Base de Datos:** MySQL (Relacional).
- **Frontend:** React.js, React Router, Axios, Context API / Redux.
- **Estándares:** API RESTful, HTTP Status Codes estándar, JWT Auth.

MODELO DE DOMINIO (atributos y relaciones)



CASOS DE USO

DIAGRAMA DE CASOS DE USO: GESTIÓN DE TALLER DE MOTOCICLETAS (UML)



Caso de uso: CU-001

CU-001	
Nombre	CU-001 Cambiar Estado de Orden de Trabajo
Actores	Mecánico, Admin
Precondiciones	El usuario ha iniciado sesión en la plataforma y se encuentra en el detalle de una orden.
Datos de entrada	- Nuevo estado - Nota/Motivo explicativo
Flujo Principal	1. El usuario selecciona el nuevo estado, ingresa una nota opcional y confirma el cambio 2. El sistema valida el token JWT del usuario (Usuario autenticado). 3. El sistema valida que el rol del usuario tenga permisos para aplicar dicho estado. 4. El sistema verifica que la transición respete la secuencia lógica (RECIBIDA → DIAGNOSTICO → EN_PROCESO → LISTA → ENTREGADA o CANCELADA). 5. El sistema confirma que el estado destino sea diferente al actual (no idempotente). 6. El sistema actualiza el estado en la orden y crea automáticamente el registro en la tabla 7. El sistema retorna un mensaje de confirmación y actualiza la vista con el historial.
Flujos Alternativos y Excepciones	- E1 (Sin JWT): Si el usuario no está autenticado, el sistema retorna error. - E2 (Rol sin permiso): Si el usuario intenta una transición no permitida para su rol (ej. Mecánico intentando pasar a ENTREGADA), el sistema retorna error 403 Forbidden. - E3 (Transición inválida): Si la secuencia no cumple el flujo permitido, el sistema retorna error 400 Bad Request. - E4 (Transición idempotente): Si el estado seleccionado es igual al estado actual, el sistema ignora la inserción en el historial y mantiene el estado sin cambios.

Caso de uso: CU-002

CU-002	
Nombre	CU-002 Crear Orden de Trabajo
Actores	Admin
Precondiciones	El usuario está autenticado con rol ADMIN en el sistema.
Datos de entrada	- Identificador o Placa de la Moto (motold / placa) - Descripción de la falla
Flujo Principal	1. El usuario selecciona/busca la moto por placa e ingresa la descripción de la falla. 2. El usuario confirma la creación de la orden. 3. El sistema valida el token JWT del usuario (Autenticado). 4. El sistema valida que el usuario posea el rol ADMIN (Autorizado). 5. El sistema verifica que la moto exista en la base de datos. 6. El sistema valida que el campo de descripción de falla no esté vacío. 7. El sistema asigna el estado inicial RECIBIDA, la fecha de ingreso actual (entryDate) y el total inicial en 0 8. El sistema guarda la orden de trabajo en la base de datos. 9. El sistema retorna respuesta exitosa (HTTP 201) y redirige o actualiza la vista hacia el detalle de la orden recién creada.
Flujos Alternativos y Excepciones	- E1 (Sin JWT): Si no hay token JWT válido, el sistema responde 401 Unauthorized. - E2 (Rol sin permiso): Si el usuario no es ADMIN, el sistema responde 403 Forbidden - E3 (Moto no existe): Si la placa de la moto no se encuentra registrada, el sistema notifica al usuario y permite ejecutar el subflujo de <i>Registro Rápido de Moto y Cliente</i>. 1. Escenario A (Moto nueva, cliente existente): La moto no existe. Se activa la extensión al CU-002a (Registrar Moto). Al crear la moto, se busca el cliente existente por su ID, se asocia y se retorna a terminar la orden. 2. Escenario B (Moto y cliente nuevos): La moto no existe → activa CU-00b. Al buscar el cliente, tampoco existe → activa CU-002b (Registrar Cliente). Se guarda el cliente, se guarda la moto asociada y finalmente se crea la orden de trabajo. - E4 (Sin Descripción de Falla): Si el campo de descripción está vacío, el sistema marca el campo en rojo y solicita al usuario completarlo antes de enviar.

Caso de uso: CU-003

CU-003	
Nombre	CU-003 Agregar Ítems a la Orden de Trabajo
Actores	Admin, Mecánico
Precondiciones	El usuario ha iniciado sesión y se encuentra en la pantalla de detalle de una orden de trabajo activa.
Datos de entrada	- Orden de trabajo id - Tipo de ítem (Mano de obra o repuesto) - Nota /descripción - Cantidad - Valor unitario
Flujo Principal	1. El usuario selecciona la opción "Agregar ítem", elige el tipo de ítem (MANO_OBRA o REPUESTO), ingresa la descripción, la cantidad, el valor unitario y confirma la creación del ítem. 2. El sistema valida el token JWT del usuario (Autenticado). 3. El sistema valida que el rol del usuario sea ADMIN o MECANICO (Autorizado). 4. El sistema valida que el tipo de ítem sea correcto (MANO_OBRA o REPUESTO). 5. El sistema valida que la descripción no esté vacía. 6. El sistema valida que la cantidad sea un número mayor a 0 (count > 0). 7. El sistema valida que el valor unitario sea mayor o igual a 0 (unitValue ≥ 0). 8. El sistema registra el nuevo ítem en la base de datos vinculado a la orden (POST /api/work-orders/:id/items) 9. El sistema recalcula y actualiza el campo total de la orden de trabajo. 10. El sistema responde HTTP 201 Created y actualiza la interfaz mostrando el nuevo ítem y el total recalculado.
Flujos Alternativos y Excepciones	- E1 (Sin JWT): Si no hay token JWT válido, el sistema responde 401 Unauthorized. - E2 (Rol sin permiso): Si el usuario no está autorizado el sistema responde 403 Forbidden - E3 (Campo obligatorio faltante): Si falta el tipo de ítem o la descripción, el sistema resalta el campo requerido y no envía la petición - E4 (Cantidad inválida): Si la cantidad está vacía o es ≤ 0, el sistema notifica el error y solicita un valor válido (Cantidad > 0). - E5 (Valor unitario inválido): Si el valor unitario está vacío o es < 0, el sistema notifica el error y solicita un valor válido (Valor unitario ≥ 0).

Caso de uso: CU-004

CU-004	
Nombre	CU-004 Eliminar Ítem de la Orden de Trabajo
Actores	Admin, Mecánico
Precondiciones	El usuario está autenticado en la plataforma y la orden de trabajo seleccionada se encuentra en un estado activo (RECIBIDA, DIAGNOSTICO, EN_PROCESO o LISTA).
Datos de entrada	- ID del ítem a eliminar (itemId)
Flujo Principal	1. El usuario selecciona la opción "Eliminar" en un ítem específico de la orden. 2. El sistema valida el token JWT del usuario (Autenticado). 3. El sistema evalúa los permisos según el rol del actor: • Si es ADMIN: Permite la eliminación de cualquier ítem. • Si es MECANICO: El sistema verifica que el ítem haya sido registrado originalmente por ese mismo mecánico (item.created_by == user.id). 4. El sistema verifica que la orden no esté en estado ENTREGADA o CANCELADA. 5. El sistema elimina el ítem de la base de datos (DELETE /api/work-orders/items/:itemId). 6. El sistema recalcula y actualiza el valor total de la orden de trabajo. 7. El sistema retorna respuesta exitosa (HTTP 200/204) y actualiza la vista removiendo el ítem y mostrando el nuevo total.
Flujos Alternativos y Excepciones	- E1 (Sin JWT): Si no hay token JWT válido, el sistema responde 401 Unauthorized. - E2 (Mecánico intenta borrar ítem ajeno): Si un usuario con rol MECANICO intenta eliminar un ítem creado por otro usuario, el sistema deniega la acción y responde 403 Forbidden. - E3 (Ítem inexistente): Si el ítem ya no existe en la base de datos, el sistema informa al usuario con un mensaje de error y refresca la lista (404 Not Found). - E4 (Orden en estado no modificable): Si la orden fue cambiada a ENTREGADA o CANCELADA, el sistema rechaza la eliminación y responde 400 Bad Request ("No se pueden modificar ítems de una orden finalizada").

Caso de uso: CU-005

CU-005	
Nombre	CU-005 Iniciar Sesión (Login)
Actores	Admin, Mecánico (Usuario del sistema)
Precondiciones	El usuario debe estar previamente registrado en la base de datos.
Datos de entrada	- Email - Contraseña
Flujo Principal	1. El usuario ingresa su correo electrónico y contraseña en el formulario de acceso y hace clic en "Iniciar Sesión". 2. El sistema busca al usuario por su email en la base de datos 3. El sistema valida que el usuario se encuentre en estado activo (active == true). 4. El sistema comprueba la contraseña utilizando bcrypt.compare contra el password_hash guardado. 5. El sistema genera el token de acceso JWT (accessToken) firmado con la clave secreta y, opcionalmente, el refreshToken. 6. El sistema responde con HTTP 200 OK, enviando el token y la información básica del usuario (sin exponer el hash de contraseña). 7. El cliente (React) almacena el token de forma segura y redirige al usuario a la pantalla principal según su rol (ADMIN o MECANICO).
Flujos Alternativos y Excepciones	- E1 (Credenciales inválidas): Si el correo no existe o la contraseña no coincide con el hash, el sistema responde 401 Unauthorized con el mensaje genérico "Correo o contraseña incorrectos" (sin especificar cuál falló). - E2 (Usuario inactivado): Si el usuario existe, pero tiene el campo active == false, el sistema bloquea el acceso y responde 403 Forbidden ("Su cuenta se encuentra inhabilitada"). - E3 (Campos requeridos incompletos): Si el usuario intenta enviar el formulario dejando el correo o la contraseña vacíos, el frontend invalida la petición y resalta los campos faltantes.

Mapa Global de Casos de Uso del Sistema

Módulo 1: Autenticación y Seguridad

- CU-005 Iniciar Sesión (Login): Autenticación de usuarios (ADMIN / MECANICO) mediante correo y contraseña, validación con bcrypt y emisión de tokens JWT corto/refresh token.

Módulo 2: Gestión de Clientes y Motos

- CU-002 Crear Orden de Trabajo: Permite al ADMIN aperturar una orden asociada a una moto y registrar la falla reportada.
 - CU-002a Registrar Motocicleta (<<extend>>): Se activa solo si la moto ingresada no existe previamente en la base de datos.
 - CU-002b Registrar Cliente (<<extend>>): Se activa si al momento de registrar la moto, el cliente tampoco existe en el sistema.

Módulo 3: Operación del Taller e Ítems

- CU-003 Agregar Ítems a la Orden: Permite a ADMIN y MECANICO añadir manos de obra o repuestos, validando cantidades/precios y recalculando el valor total.
- CU-004 Eliminar Ítem de la Orden: Permite desvincular un ítem y ajustar el total. Aplica permisos diferenciados: ADMIN elimina cualquiera, mientras que MECANICO solo puede eliminar los creados por él mismo.

Módulo 4: Flujo de Estados e Historial

- CU-001 Cambiar Estado de Orden de Trabajo: Controla la transición secuencial de estados (RECIBIDA → DIAGNOSTICO → EN_PROCESO → LISTA → ENTREGADA o CANCELADA), valida permisos por rol y guarda automáticamente el registro auditado en el historial (work_order_status_history).

DISEÑO DE REGLAS DE NEGOCIO

Matriz de Transición de Estado

Estado Actual (Fila) \ Estado Destino (Columna)	RECIBIDA	DIAGNOSTICO	EN_PROCESO	LISTA	ENTREGADA	CANCELADA
RECIBIDA	NO	SI	NO	NO	NO	SI
DIAGNOSTICO	NO	NO	SI	NO	NO	SI
EN_PROCESO	NO	NO	NO	SI	NO	SI
LISTA	NO	NO	NO	NO	SI	SI
ENTREGADA	NO	NO	NO	NO	NO	NO
CANCELADA	NO	NO	NO	NO	NO	NO

Matriz de Control de Acceso por Roles (RBAC)

Acción / Recurso	ADMIN	MECANICO
Crear / Editar Clientes y Motos	SÍ	NO
Crear Orden de Trabajo	SÍ	NO
Agregar Ítems a Orden	SÍ	SÍ
Eliminar Ítems (Cualquiera)	SÍ	NO
Eliminar Ítems (Propios)	SÍ	SÍ
Cambiar estado a DIAGNOSTICO, EN_PROCESO, LISTA	SÍ	SÍ
Cambiar estado a ENTREGADA o CANCELADA	SÍ	NO
Gestionar Usuarios (Crear, cambiar rol, desactivar)	SÍ	NO

Tabla de Invariantes y Validaciones de Campo

Entidad / Campo	Regla / Invariante	Tipo de Validación	Error Retornado
Moto → Placa	Debe ser única en la BD (UNIQUE).	Integridad de BD / API	400 Bad Request ("La placa ya existe").
Ítem → Cantidad	Valor numérico entero > 0.	Validación de Entrada	400 Bad Request ("La cantidad debe ser mayor a 0").
Ítem → Valor Unitario	Valor numérico ≥ 0.	Validación de Entrada	400 Bad Request ("El valor no puede ser negativo").
Orden → Total	$\sum (\text{count} \times \text{unitValue})$	Calculado / Invariante	Automático por Backend.
Historial → Transición	Bloquear si from_status == to_status.	Lógica de Negocio	Se ignora el registro (Idempotencia).

Reglas de Negocio Consolidadas (RN)

1. Reglas de Ciclo de Vida y Estados (Matriz de Transición)

- RN-01 (Flujo Secuencial): Una orden de trabajo solo puede avanzar de forma lineal: RECIBIDA → DIAGNOSTICO → EN_PROCESO → LISTA → ENTREGADA. No se permite el salto de estados.
- RN-02 (Regla de Cancelación): El estado CANCELADA es accesible desde cualquier estado activo, excepto cuando la orden ya está en estado ENTREGADA.
- RN-03 (Inmutabilidad de Estados Finales): Los estados ENTREGADA y CANCELADA son terminales. Ninguna orden en estos estados puede ser reabierta o modificada.

2. Reglas de Autorización y Roles (Matriz RBAC)

- RN-04 (Restricción de Cierre de Orden): Solo el rol ADMIN tiene la potestad de autorizar el cambio de estado hacia ENTREGADA o CANCELADA.

- RN-05 (Propiedad de Eliminación de Ítems): El rol ADMIN puede eliminar cualquier ítem de una orden. El rol MECANICO únicamente puede eliminar los ítems que él mismo haya registrado (`created_by == user_id`).
- RN-06 (Gestión Exclusiva de Entidades): Solo el rol ADMIN puede aperturar órdenes de trabajo, registrar clientes/vehículos y gestionar cuentas de usuario (ADMIN / MECANICO).

3. Reglas de Integridad y Fórmulas (Invariantes y Validaciones)

- RN-07 (Unicidad de Identificadores): La placa de cada vehículo debe ser única en todo el sistema.
- RN-08 (Valores Financieros y Operativos): Todo ítem adjunto a una orden debe tener una cantidad estrictamente mayor a cero ($\text{cantidad} > 0$) y un valor unitario no negativo ($\text{valor} \geq 0$).
- RN-09 (Recálculo Automático del Total): El total de la orden de trabajo es una propiedad calculada e invariante: $\text{Total} = \sum(\text{cantidad} \times \text{valor_unitario})$. Debe actualizarse automáticamente tras agregar o eliminar un ítem.
- RN-10 (Trazabilidad e Inmutabilidad de Auditoría): Cada transición de estado debe generar de manera automática un registro inalterable en la tabla de historial capturando: estado origen, estado destino, usuario responsable, nota y fecha/hora. Se descartan transiciones hacia el mismo estado (idempotencia).

VALIDACIÓN DEL MODELO

Matriz de Validación y Trazabilidad del Modelo

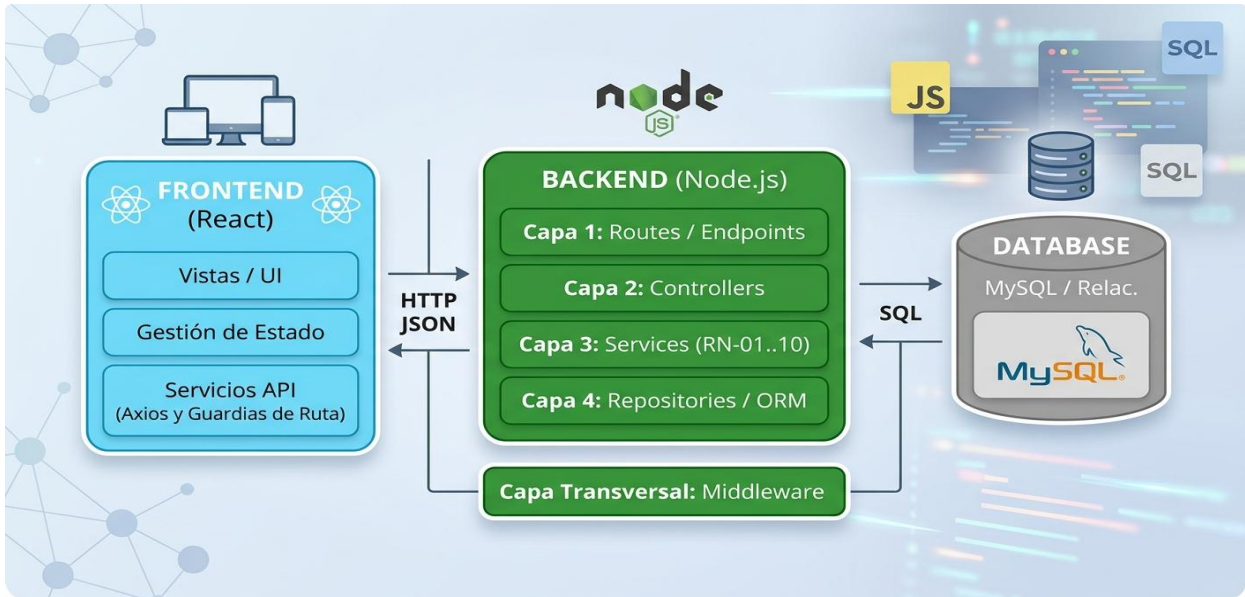
Regla de Negocio (RN)	Descripción de la Regla	Casos de Uso que la Implementan	Sección / Paso Específico donde se Evalúa	Estado de Cobertura
RN-01	Flujo Secuencial de Estados: Transición lineal obligatoria (RECIBIDA → DIAGNOSTICO → EN_PROCESO → LISTA → ENTREGADA).	CU-001, CU-002	CU-001: Paso 4, Excepción E3. CU-002: Paso 7 (Estado inicial).	Cubierta (100%)
RN-02	Regla de Cancelación: Permitida desde cualquier estado activo, excepto si la orden ya fue entregada.	CU-001	CU-001: Paso 4, Excepción E3.	Cubierta (100%)
RN-03	Inmutabilidad de Estados Terminales: Bloqueo total de modificaciones si la orden está en ENTREGADA o CANCELADA.	CU-001, CU-003, CU-004	CU-001: Excepción E3. CU-003: Precondición. CU-004: Precondición, Paso 4, Excepción E4.	Cubierta (100%)
RN-04	Restricción de Cierre de Orden: Exclusividad del rol ADMIN para autorizar cambios a ENTREGADA o CANCELADA.	CU-001	CU-001: Paso 3, Excepción E2.	Cubierta (100%)
RN-05	Propiedad de Eliminación de Ítems: ADMIN elimina cualquiera; MECANICO	CU-004	CU-004: Paso 3, Excepción E2.	Cubierta (100%)

	únicamente los creados por él mismo (created_by == user.id).			
RN-06	Gestión Exclusiva de Entidades y Seguridad RBAC: Operaciones administrativas y acceso restringido por roles vía tokens JWT.	CU-001, CU-002, CU-003, CU-004, CU-005	CU-005: Pasos 3-5, Excepciones E1 y E2. Transversal en todos los CU (Manejo de JWT y Roles).	Cubierta (100%)
RN-07	Unicidad de Identificadores: La placa de cada vehículo debe ser única en el sistema (UNIQUE).	CU-002	CU-002: Excepción E3 (Subflujo de Registro Rápido).	Cubierta (100%)
RN-08	Valores Financieros y Operativos: Ítems requieren cantidad > 0 y valor_unitario ≥ 0.	CU-003	CU-003: Pasos 6 y 7, Excepciones E4 y E5.	Cubierta (100%)
RN-09	Recálculo Automático del Total: Total como propiedad calculada e invariante: Total = $\sum(\text{cantidad} \times \text{valor_unitario})$.	CU-002, CU-003, CU-004	CU-002: Paso 7 (Inicial en 0). CU-003: Pasos 9 y 10. CU-004: Pasos 6 y 7.	Cubierta (100%)
RN-10	Trazabilidad e Inmutabilidad de Auditoría: Registro automático de historial en cambios de estado (con idempotencia).	CU-001	CU-001: Pasos 5 y 6, Excepción E4.	Cubierta (100%)

DISEÑO DE LA ARQUITECTURA

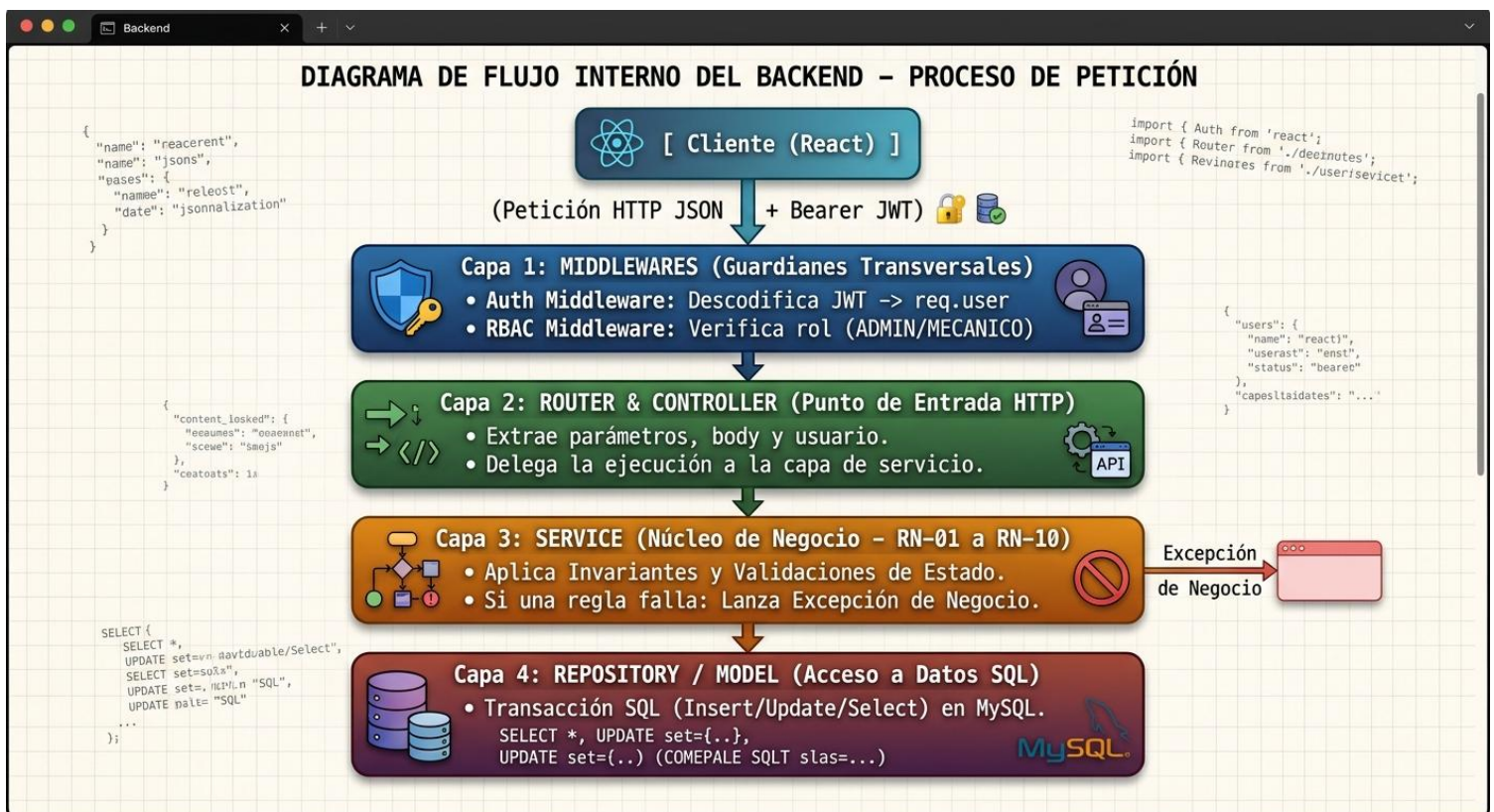
La Arquitectura Global del Sistema.

A nivel global, la arquitectura de nuestro proyecto responde al patrón Cliente-Servidor desacoplado (Single Page Application + REST API):

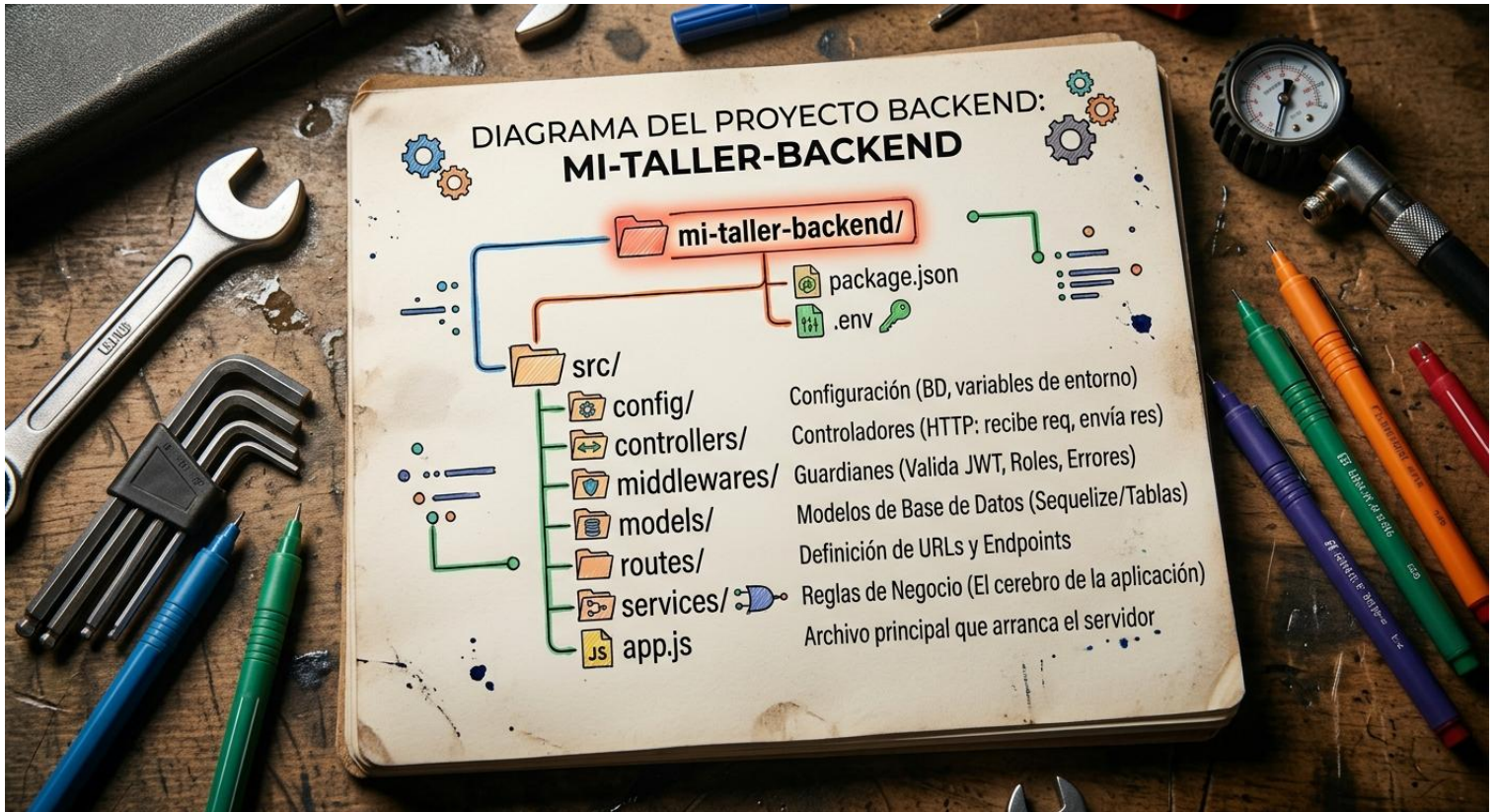


Diseño Técnico de la Arquitectura del Backend.

Definición de Capas (Flujo Interno del Backend)



La Estructura Base de Carpetas para Backend



Módulo: Iniciar Sesión (CU-005)

Mapeo del Módulo Auth en la Arquitectura

Para implementar estos 5 endpoints, el Backend solo va a tocar 3 archivos principales en las carpetas correspondientes:

1. src/routes/auth.routes.js (*Definición de URLs y accesos*)
2. src/controllers/auth.controller.js (*Manejo de peticiones HTTP*)
3. src/services/auth.service.js (*Lógica de Negocio y Seguridad*)

1. La Capa de Rutas (src/routes/auth.routes.js)

- POST /api/auth/register:
 - Filtro / Middleware: Requiere estar autenticado y ser ADMIN (RN-06).
 - Destino: authController.register

- POST /api/auth/login:
 - Filtro / Middleware: Ninguno (es público).
 - Destino: authController.login (CU-005).
- POST /api/auth/refresh:
 - Filtro / Middleware: Valida que envíe un Refresh Token válido.
 - Destino: authController.refreshToken
- POST /api/auth/logout:
 - Filtro / Middleware: Valida que el usuario tenga sesión activa.
 - Destino: authController.logout
- GET /api/auth/me:
 - Filtro / Middleware: Valida token JWT activo (authMiddleware).
 - Destino: authController.getProfile

2. La Capa de Controladores (src/controllers/auth.controller.js)

El controlador no contiene lógica compleja: recibe la solicitud, extrae los datos y responde.

Miremos el ejemplo específico del POST /api/auth/login (CU-005):

1. Extrae el email y password desde el cuerpo de la petición (req.body).
2. Llama a la capa de servicio: authService.loginUser(email, password).
3. Cuando el servicio responde con el token y el usuario, el controlador responde con HTTP 200 OK y el JSON correspondiente.
4. Si algo falla (ej. contraseña incorrecta), el controlador deja que el error fluya al Middleware Global de Errores.

3. La Capa de Servicios (src/services/auth.service.js)

Este archivo ejecuta los pasos exactos que definidos en el CU-005:

- Para el Login (loginUser):
 1. Busca en la base de datos si existe el usuario con ese email.
 2. Evalúa si el usuario está activo (active == true). Si está inactivo, rechaza la operación (Excepción E2).
 3. Compara la contraseña ingresada contra el hash encriptado usando bcrypt. Si no coincide, rechaza con error genérico (Excepción E1).
 4. Firma y genera el token de acceso JWT con el id y el role del usuario.
 5. Retorna el token y los datos seguros del usuario.

Módulo: Módulo Órdenes + Ítems (CU-001, CU-002, CU-003, CU-004)

POST /api/work-orders

- Ubicación: routes/workOrder.routes.js → controllers/workOrder.controller.js → services/workOrder.service.js
- Funcionalidad: Crea la orden de trabajo inicial en estado RECIBIDA (**CU-002**).

GET /api/work-orders?status=&plate=&page=&pageSize=

- Ubicación: routes/workOrder.routes.js → controllers/workOrder.controller.js → services/workOrder.service.js
- Funcionalidad: Consulta el listado general de órdenes con filtros de búsqueda y paginación.

GET /api/work-orders/:id

- Ubicación: routes/workOrder.routes.js → controllers/workOrder.controller.js → services/workOrder.service.js
- Funcionalidad: Trae el detalle completo de una orden específica (incluye sus ítems e historial de estados).

PATCH /api/work-orders/:id/status

- Ubicación: routes/workOrder.routes.js → controllers/workOrder.controller.js → services/workOrder.service.js

- Funcionalidad: Actualiza el estado de la orden respetando la secuencia permitida (CU-001).

POST /api/work-orders/:id/items

- Ubicación: routes/item.routes.js → controllers/item.controller.js → services/item.service.js
- Funcionalidad: Agrega repuestos o mano de obra y recalcula el total de la orden (CU-003).

DELETE /api/work-orders/items/:itemId

- Ubicación: routes/item.routes.js → controllers/item.controller.js → services/item.service.js
- Funcionalidad: Elimina un ítem específico verificando permisos y recalcula el total (CU-004).

GET /api/work-orders/:id/history

- Ubicación: routes/workOrder.routes.js → controllers/workOrder.controller.js → services/workOrder.service.js
- Funcionalidad: Consulta la traza de auditoría de una orden, retornando la lista de cambios de estado ordenados cronológicamente (createdAt DESC) para saber quién cambió el estado, cuándo y por qué nota.

PATCH /api/work-orders/:id/status *(Ajuste en la firma del cuerpo)*

- Ubicación: routes/workOrder.routes.js → controllers/workOrder.controller.js → services/workOrder.service.js
- Estructura del Body: { "toStatus": "DIAGNOSTICO", "note": "Se identificó fuga en retén de suspensión" }
- Funcionalidad: Ejecuta la transición de estado (RN-01, RN-03, RN-04) y en la misma transacción inserta el registro correspondiente en la tabla de historial (RN-10).

Módulo: Motos (CU-002a)

- POST /api/bikes
 - Ubicación: routes/bike.routes.js → controllers/bike.controller.js → services/bike.service.js
 - Funcionalidad: Registra una nueva motocicleta verificando la unicidad de la placa (RN-07).
- GET /api/bikes?plate=
 - Ubicación: routes/bike.routes.js → controllers/bike.controller.js → services/bike.service.js
 - Funcionalidad: Busca una motocicleta registrada por su número de placa.
- GET /api/bikes/:id
 - Ubicación: routes/bike.routes.js → controllers/bike.controller.js → services/bike.service.js
 - Funcionalidad: Obtiene el detalle técnico de una motocicleta específica por su ID.

Módulo: Cliente (CU-002b)

POST /api/clients

- Ubicación: routes/client.routes.js → controllers/client.controller.js → services/client.service.js
- Funcionalidad: Registra a un nuevo cliente (dueño del vehículo) en el sistema.

GET /api/clients?search=

- Ubicación: routes/client.routes.js → controllers/client.controller.js → services/client.service.js
- Funcionalidad: Busca clientes por nombre, teléfono o documento de identidad.

GET /api/clients/:id

- Ubicación: routes/client.routes.js → controllers/client.controller.js → services/client.service.js

- Funcionalidad: Obtiene el perfil completo y datos de contacto de un cliente específico.

Estrategia Global de Errores

Regla de Negocio / Escenario	Excepción de Negocio	Código HTTP	Mensaje Estándar JSON
RN-01: Transición de estado no permitida en la secuencia (ej. RECIBIDA → ENTREGADA).	InvalidStateTransitionError	400 Bad Request	"Transición de estado no permitida en el flujo de trabajo."
RN-02: Intento de entregar orden con ítems pendientes de reparación.	PendingItemsError	400 Bad Request	"No se puede entregar la orden porque existen ítems sin finalizar."
RN-03: Intentar modificar o agregar ítems a una orden en estado ENTREGADA o CANCELADA.	OrderClosedError	400 Bad Request	"La orden se encuentra finalizada o cancelada y no admite cambios."
RN-04: Intento por parte de un MECANICO de pasar a ENTREGADA o CANCELADA.	ForbiddenActionError	403 Forbidden	"Requiere privilegios de Administrador para cerrar o cancelar la orden."
RN-05: Mecánico intentando eliminar un ítem creado por otro usuario (created_by != user.id).	ItemOwnershipError	403 Forbidden	"No tienes permisos para eliminar este ítem."
RN-06: Intento de crear usuarios o registrar repuestos/servicios sin rol ADMIN.	AdminOnlyError	403 Forbidden	"Acceso denegado. Esta acción requiere rol de Administrador."
RN-07: Registro de motocicleta con una placa que ya existe en la base de datos.	DuplicatePlateError	409 Conflict	"La placa ingresada ya se encuentra registrada en el sistema."

RN-08: Cantidad ≤ 0 o valor unitario < 0 al agregar un ítem.	InvalidItemDataError	400 Bad Request	"La cantidad y el valor unitario deben ser montos válidos superiores a cero."
RN-09: Fallo en el recálculo o inconsistencia en el monto total de la orden.	CalculationError	400 Bad Request	"Error en el cálculo del total acumulado de la orden."
RN-10: Intento de modificar o eliminar un registro del historial de auditoría de estados.	AuditImmutableError	403 Forbidden	"El historial de auditoría de estados es inmutable y no puede alterarse."
General (CU-005): Token JWT ausente, expirado o manipulado en la petición HTTP.	UnauthorizedError	401 Unauthorized	"Credenciales de acceso inválidas o sesión expirada."
General: Buscar una Orden, Moto, Cliente o Ítem con un ID que no existe en MySQL.	ResourceNotFoundError	404 Not Found	"El recurso solicitado no fue encontrado en el sistema."

Estructura general de carpetas

mi-taller-backend/

```

├── .env                # Variables de entorno (Puerto, Secret Key JWT, Credenciales
BD)
├── .gitignore          # Archivos ignorados por Git (node_modules, .env)
├── package.json        # Dependencias del proyecto
├── README.md           # Documentación técnica
|
└── src/
    ├── app.js          # Configuración global de Express y montaje de rutas
    ├── server.js        # Punto de entrada que levanta el servidor HTTP
    |
    └── config/          # CAPA DE CONFIGURACIÓN

```

- | └─ db.js # Conexión a la base de datos MySQL
- | └─ env.js # Validación y carga de variables de entorno
- |
- | └─ **routes/ # CAPA DE RUTAS (VENTANILLAS HTTP)**
- | └─ index.routes.js # Enrutador principal (prefijo /api/v1)
- | └─ auth.routes.js # Endpoints CU-005 (login, register, refresh, logout, me)
- | └─ workOrder.routes.js # Endpoints CU-001, CU-002, CU-003, CU-004
(órdenes, ítems, historial)
- | └─ bike.routes.js # Endpoints CU-002a (motos: crear, buscar por placa,
detalle)
- | └─ client.routes.js # Endpoints CU-002b (clientes: crear, buscar, detalle)
- |
- | └─ **controllers/ # CAPA DE CONTROLADORES (CONSERJES HTTP)**
- | └─ auth.controller.js
- | └─ workOrder.controller.js
- | └─ bike.controller.js
- | └─ client.controller.js
- |
- | └─ **services/ # CAPA DE SERVICIOS (EL CEREBRO / REGLAS DE
NEGOCIO)**
- | └─ auth.service.js # Encriptación bcrypt, generación JWT
- | └─ workOrder.service.js # Evaluador de RN-01, RN-02, RN-03, RN-04, RN-09,
RN-10
- | └─ item.service.js # Evaluador de RN-05 (dueño ítem), RN-08 (montos
validos)
- | └─ bike.service.js # Evaluador de RN-07 (placa única)
- | └─ client.service.js

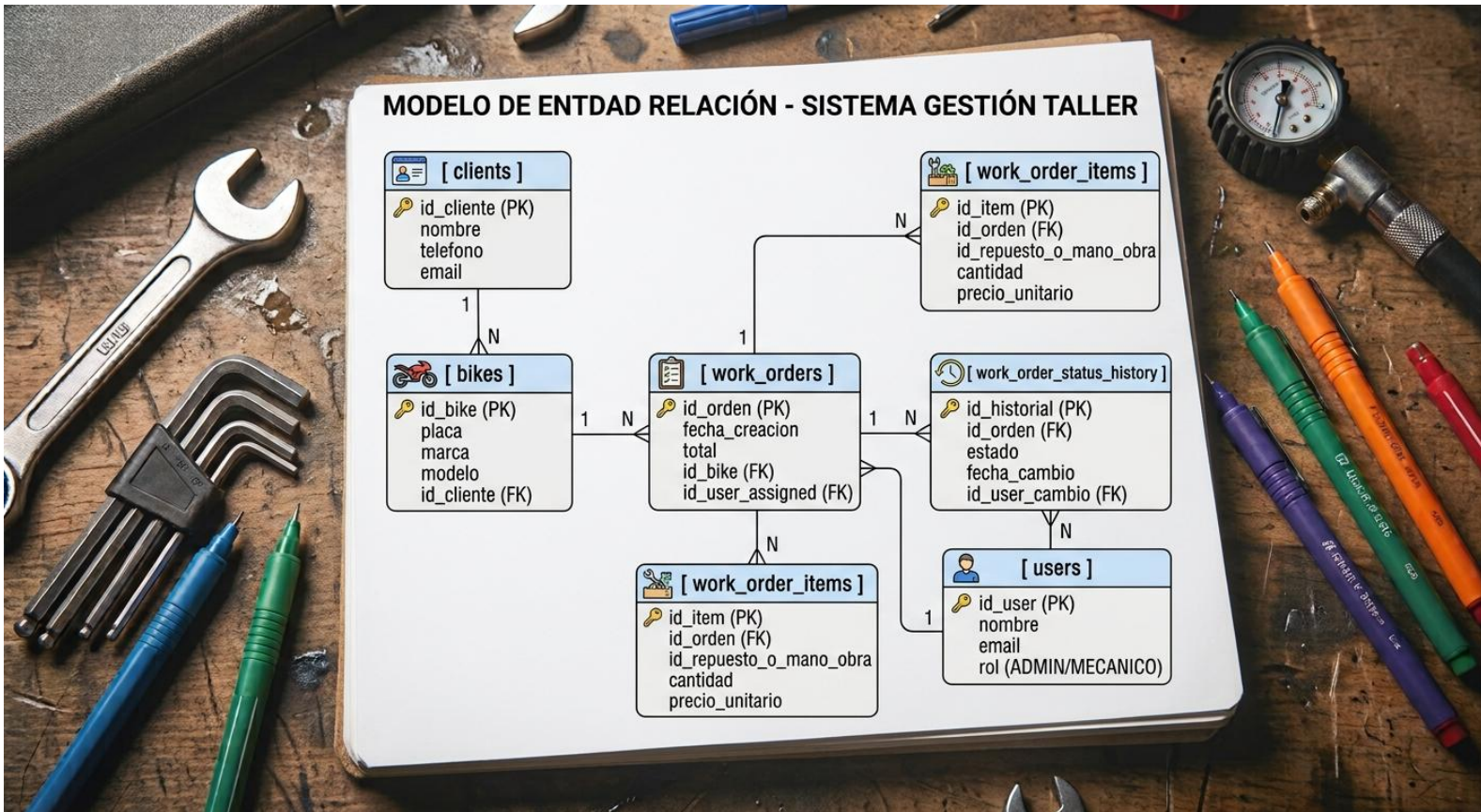
```

|
| └─ models/          # CAPA DE DATOS (REPOSITORIOS / MODELOS SQL)
|   | └─ User.js      # Entidad Usuarios y Roles (ADMIN / MECANICO)
|   | └─ Client.js    # Entidad Clientes
|   | └─ Bike.js      # Entidad Motocicletas
|   | └─ WorkOrder.js  # Entidad Órdenes de Trabajo
|   | └─ WorkOrderItem.js # Entidad Ítems (Repuestos / Mano de Obra)
|   └─ StatusHistory.js # Entidad Auditoría de Estados (Inmutable RN-10)
|
| └─ middlewares/      # CAPA DE GUARDIANES Y TRANSVERSALES
|   | └─ auth.middleware.js # Verificación y decodificación del Token JWT
|   | └─ rbac.middleware.js # Control de acceso basado en roles (RN-04, RN-06)
|   └─ error.middleware.js # Middleware Global de Errores (Atrapamoscas RFC
7807)
|
| └─ utils/            # HERRAMIENTAS Y CLASES DE APOYO
|   | └─ errors.js     # Clases personalizadas (BadRequestError, ForbiddenError,
etc.)
|   └─ responseHandler.js # Formateador de respuestas exitosas unificadas

```

DISEÑO DE LA BASE DE DATOS

Modelo entidad relación



Relaciones

- Un Cliente (clients) puede tener muchas Motos (bikes).
- Una Moto (bikes) puede tener muchas Órdenes de Trabajo (work_orders).
- Una Orden de Trabajo (work_orders) tiene muchos Ítems (work_order_items) (Repuestos/Mano de obra).
- Una Orden de Trabajo (work_orders) guarda muchos registros en su Historial de Estados (work_order_status_history).
- Un Usuario (users) (Mecánico o Admin) registra las acciones y los cambios de estado en la auditoría.

Análisis de Tipos de Datos y Restricciones (Constraints)

Para que la base de datos proteja la integridad de la información por sí sola (incluso si el código falla), definiremos tipos de datos precisos para MySQL:

Tabla users (Módulo Autenticación - CU-005)

- id: BIGINT AUTO_INCREMENT PRIMARY KEY
- email: VARCHAR(150) NOT NULL UNIQUE → No pueden existir dos usuarios con el mismo correo.
- password_hash: VARCHAR(255) NOT NULL → Aquí se guarda la clave encriptada con bcrypt.
- role: ENUM('ADMIN', 'MECANICO') NOT NULL → Solo permite estos dos valores estrictos para la RN-04 y RN-06.
- active: BOOLEAN DEFAULT TRUE → Para la Excepción E2 del login.

Tabla bikes (Módulo Motos – CU-002^a)

- placa: VARCHAR(10) NOT NULL UNIQUE → **Cumple la RN-07:** La base de datos arroja error si intentas registrar una placa repetida.
- client_id: BIGINT NOT NULL → FOREIGN KEY REFERENCES clients(id) ON DELETE RESTRICT.

Tabla work_orders (Módulo Órdenes - CU-001 y CU-002)

- status: ENUM('RECIBIDA', 'DIAGNOSTICO', 'EN_REPARACION', 'LISTA', 'ENTREGADA', 'CANCELADA') DEFAULT 'RECIBIDA' → Los 6 estados oficiales del flujo.
- total: DECIMAL(12, 2) DEFAULT 0.00 → Usamos DECIMAL y NO FLOAT/DOUBLE para evitar errores de redondeo con el dinero.

Tabla work_order_items (Módulo Ítems – CU-003 y CU-004)

- type: ENUM('MANO_OBRA', 'REPUESTO') NOT NULL
- count: INT NOT NULL → Con restricción CHECK (count > 0) (**RN-08**).
- unit_value: DECIMAL(12, 2) NOT NULL → Con restricción CHECK (unit_value >= 0) (**RN-08**).
- created_by_user_id: BIGINT NOT NULL → **Vital para la RN-05:** Guarda la ID del usuario que creó el ítem para verificar si puede borrarlo.

Tabla `work_order_status_history` (Auditoría – RN-10)

- `from_status`: VARCHAR(30) NULL (Es NULL solo en el primer registro cuando la orden recién se crea en RECIBIDA).
- `to_status`: VARCHAR(30) NOT NULL
- `changed_by_user_id`: BIGINT NOT NULL → FOREIGN KEY REFERENCES `users(id)`.
- Índice Compuesto: INDEX `idx_order_history` (`work_order_id`, `created_at` DESC)
→ Optimización SQL: Permite que la consulta GET `/api/work-orders/:id/history` traiga los cambios en milisegundos sin alentar la BD.

Script SQL Oficial de Creación (DDL para MySQL)

```
CREATE DATABASE IF NOT EXISTS taller_db;  
USE taller_db;
```

-- 1. TABLA USUARIOS

```
CREATE TABLE IF NOT EXISTS users (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  role ENUM('ADMIN', 'MECANICO') NOT NULL,  
  active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP  
);
```

-- 2. TABLA CLIENTES

```
CREATE TABLE IF NOT EXISTS clients (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  phone VARCHAR(20) NOT NULL,  
  email VARCHAR(150) NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP  
);
```

-- 3. TABLA MOTOS (RN-07)

```
CREATE TABLE IF NOT EXISTS bikes (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  plate VARCHAR(10) NOT NULL UNIQUE,  
  brand VARCHAR(50) NOT NULL,  
  model VARCHAR(50) NOT NULL,  
  cylinder VARCHAR(20) NULL,  
  client_id BIGINT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP  DEFAULT  CURRENT_TIMESTAMP  ON  UPDATE  
CURRENT_TIMESTAMP,  
  FOREIGN KEY (client_id) REFERENCES clients(id) ON DELETE RESTRICT  
);
```

-- 4. TABLA ÓRDENES DE TRABAJO

```
CREATE TABLE IF NOT EXISTS work_orders (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  bike_id BIGINT NOT NULL,  
  entry_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  fault_description TEXT NOT NULL,  
  status  ENUM('RECIBIDA', 'DIAGNOSTICO', 'EN_REPARACION', 'LISTA',  
'ENTREGADA', 'CANCELADA') DEFAULT 'RECIBIDA',  
  total DECIMAL(12, 2) DEFAULT 0.00,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at  TIMESTAMP  DEFAULT  CURRENT_TIMESTAMP  ON  UPDATE  
CURRENT_TIMESTAMP,  
  FOREIGN KEY (bike_id) REFERENCES bikes(id) ON DELETE RESTRICT  
);
```

-- 5. TABLA ÍTEMS DE ÓRDENES (RN-05, RN-08, RN-09)

```
CREATE TABLE IF NOT EXISTS work_order_items (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  work_order_id BIGINT NOT NULL,  
  type ENUM('MANO_OBRA', 'REPUESTO') NOT NULL,  
  description VARCHAR(255) NOT NULL,  
  count INT NOT NULL CHECK (count > 0),  
  unit_value DECIMAL(12, 2) NOT NULL CHECK (unit_value >= 0),
```

```

    created_by_user_id BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (work_order_id) REFERENCES work_orders(id) ON DELETE
    CASCADE,
    FOREIGN KEY (created_by_user_id) REFERENCES users(id) ON DELETE RESTRICT
);

```

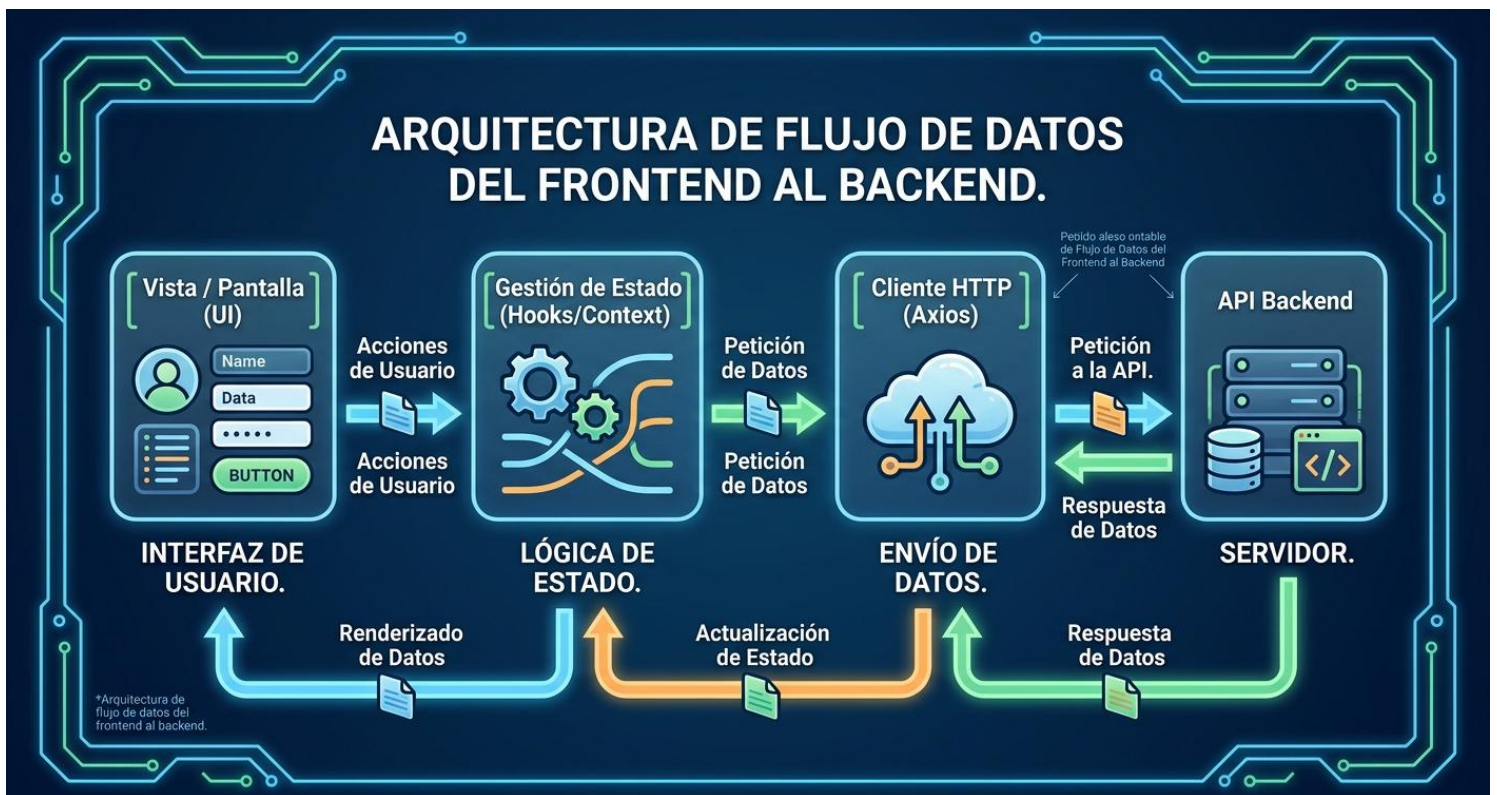
-- 6. TABLA HISTORIAL DE ESTADOS (RN-10)

```

CREATE TABLE IF NOT EXISTS work_order_status_history (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    work_order_id BIGINT NOT NULL,
    from_status VARCHAR(30) NULL,
    to_status VARCHAR(30) NOT NULL,
    note TEXT NULL,
    changed_by_user_id BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (work_order_id) REFERENCES work_orders(id) ON DELETE
    CASCADE,
    FOREIGN KEY (changed_by_user_id) REFERENCES users(id) ON DELETE
    RESTRICT,
    INDEX idx_work_order_history (work_order_id, created_at DESC)
);

```

ARQUITECTURA DE FRONTEND



Estructura Físicas de Carpetas en React

mi-taller-frontend/

```
|— public/          # Archivos estáticos (favicon, imágenes)
|— src/
|   |— assets/      # Estilos globales, logos, íconos
|   |
|   |— config/      # CONFIGURACIÓN GLOBAL
|   |   └─ axios.js  # Cliente HTTP pre-configurado con la URL base del Backend
|   |
|   |— context/      # ESTADO GLOBAL DE LA APLICACIÓN
|   |   └─ AuthContext.jsx # Guarda la sesión activa, el token JWT y el rol del
```

usuario

```
|   |
|   |— services/     # CAPA DE CONEXIÓN CON EL BACKEND (API REST)
|   |   └─ auth.service.js # Funciones login(), refreshToken(), logout()
|   |       └─ workOrder.service.js # Funciones getOrders(), createOrder(),
```

updateStatus()

```
|   |   └─ bike.service.js # Funciones getBikeByPlate(), createBike()
|   |       └─ client.service.js # Funciones searchClients(), createClient()
|   |
|   |— components/    # COMPONENTES VISUALES REUTILIZABLES (Átomos)
|   |   └─ ui/         # Botones, Inputs, Modales, Módulos de alerta
|   |       └─ layout/  # Navbar, Sidebar, Contenedor principal
|   |           └─ ProtectedRoute.jsx # Guardián que bloquea pantallas si no estás
```

logueado

```
|   |
|   |— pages/         # VISTAS / PANTALLAS PRINCIPALES (Casos de Uso)
|   |   └─ LoginPage.jsx # Pantalla de Login (CU-005)
|   |       └─ DashboardPage.jsx # Vista general del taller
|   |           └─ WorkOrdersPage.jsx # Tabla con listado de órdenes y filtros
|   |               └─ WorkOrderDetailPage.jsx # Detalle de la orden, ítems (CU-003/004) e
```

historial (CU-001)

```
|   |
|   |— routes/        # ENRUTAMIENTO DE LA APLICACIÓN
|   |   └─ AppRouter.jsx # Mapas de URLs del navegador (/login, /orders, etc.)
```



El Mapa de Pantallas (Relación Pantalla vs Endpoints)

Pantalla (Page)	Propósito en el Negocio	Endpoints que consume
LoginPage	Iniciar sesión en el sistema.	POST /api/auth/login
WorkOrdersPage	Ver el listado de órdenes de trabajo con buscador y filtros por estado o placa.	GET /api/work-orders?status=&plate=&page=
CreateOrderPage	Registrar la entrada de una moto al taller. Permite buscar/crear cliente y moto al mismo tiempo.	GET /api/clientes?search= POST /api/clientes GET /api/bikes?plate= POST /api/bikes POST /api/work-orders
WorkOrderDetailPage	La pantalla estrella del taller. Muestra los datos de la moto, permite agregar repuestos/mano de obra, cambiar el estado y ver la traza de auditoría.	GET /api/work-orders/:id POST /api/work-orders/:id/items DELETE /api/work-orders/items/:itemId PATCH /api/work-orders/:id/status GET /api/work-orders/:id/history

La Pieza Clave del Frontend: Manejo Automático del Token JWT

Como el token vence, el usuario no debería estar metiendo la clave cada 15 minutos.

El Frontend maneja esto con un Interceptor de Axios en `src/config/axios.js`:

